

NoSQL databases

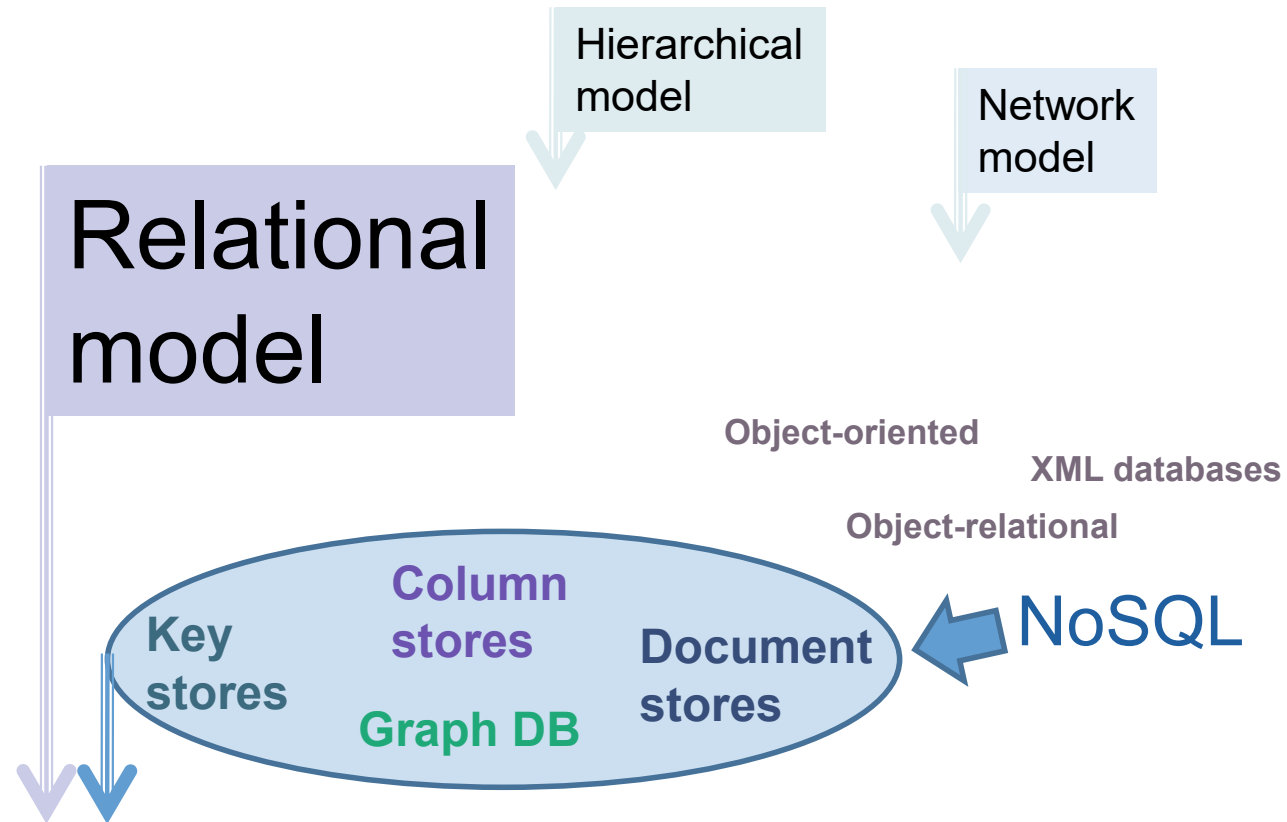
Daniele Riboni

Università degli Studi di Cagliari

Dept. of Mathematics and Computer Science

Storia dei modelli logici per DBMS

- 1960s
- 1970s
- 1980s
- 1990s
- 2000s
- 2010s
- 2020s



NoSQL

- A family of DBMS that **do not use the relational model**
- They don't rely on tables!
- They use different logical models:
 - <key, value> pairs
 - “documents” (JSON, XML, other ones...)
 - graphs

Motives and main drivers

- “Big Data” explosion
 - Massive and complex datasets
 - 3Vs: Volume, Velocity, Variety
- Relational DBMS are not intended for big data



Ex. 1: Sensor data from “smart cities”

-

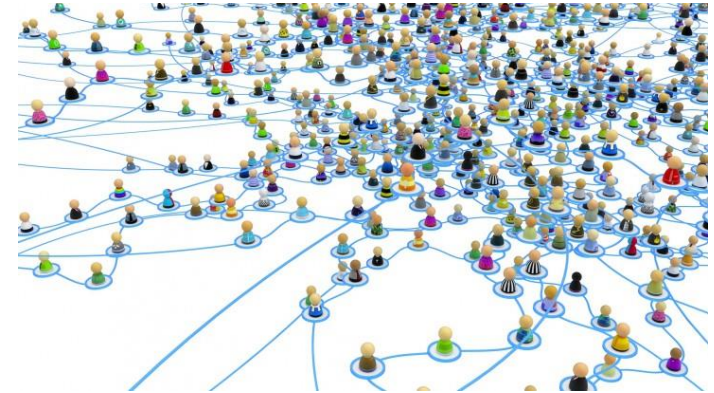
Example 2: Tweets



- Data volume (2017):
6,000 tweets per second – 500 million per day
200 billion per year
- Tweet record:
UserID, timestamp, text, additional info...
- Tasks: **Add tweet**; Change tweet status;
Retrieve tweets given some search key,
hashtag, etc

Example 3: Social-network graphs

- Friendship record:
 <UserID1, UserID2>
- User records:
 UserID, name, age, gender, ...
- Task: Find all friends
 of a friend
 of a friend ...
 of a given user



Relational DBMS and Big Data

- The relational model is based on values and datatypes → **Big Data are variegate and flexible**
- Queries are based on **JOIN** → **too expensive**
- Queries are based on **rigid syntax** → big data want “Google-like” queries
- A lot of effort for **reliability and concurrency** → not as crucial for some big data applications

Requirements for Big Data handling

- Avoid unneeded complexity to achieve scalability
- Simple schemas (or no schema at all)
- Trading reliability for performance
- Data structures closer to programming than to DBMS
 - “More coding, less DB design”

Scalability through distribution

- **Vertical scaling**: adding more power (CPU, RAM, etc.) to your existing machine
 - Scalability is limited
- **Horizontal scaling**: adding more machines to your system
 - Through distribution and parallelism
 - But we have seen that it's hard to achieve consistency and reliability!

Horizontal scaling at Google



Server racks in a Google datacenter



CAP properties of distributed systems

- **Consistency**
 - At any instant, all nodes see the same data
- **Availability**
 - The system always serves requests
- **Partition tolerance**
 - The system continues to function even if the communication among the servers is unreliable, or in case of nodes failure

CAP Theorem

“It is not possible to fulfill all three CAP properties in a distributed system”

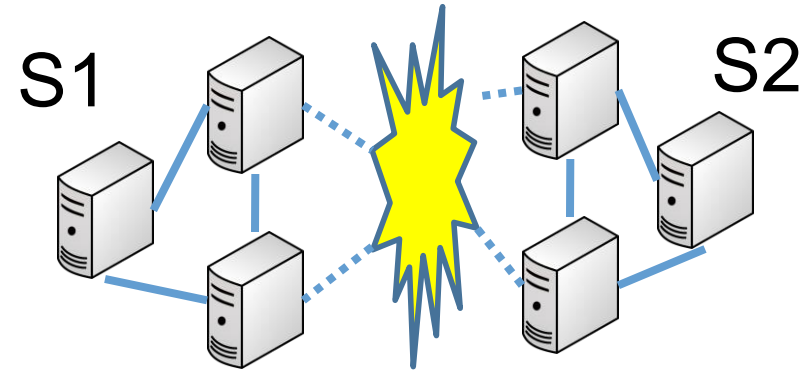
Eric Brewer, 1999

- If the DBMS must be partition tolerant (P), we must choose between C and A
- C can be replaced by “eventual consistency”: all replicas will converge to the same state after network connectivity has been re-established

CAP Theorem

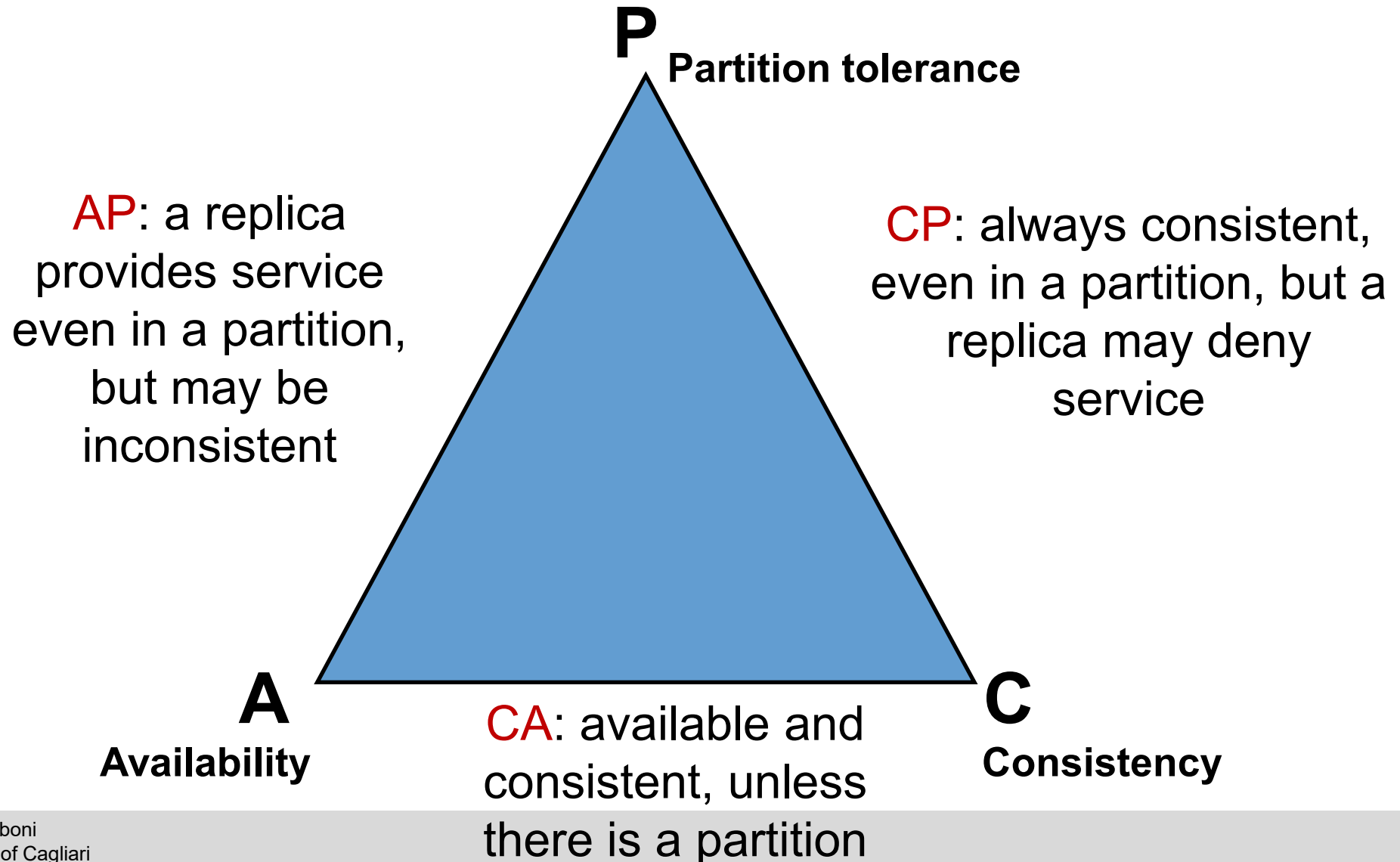
Interpretation:

- Imagine a database S partitioned in 2 sets of nodes $S1$, $S2$ that cannot communicate between each other
- If it denies all write requests in both $S1$ and $S2$, it is consistent but not available
- If $S1$ and $S2$ accept write requests, S is available but possibly inconsistent



CAP theorem

Claim: every distributed system is on one side of the triangle



ACID vs BASE

- BASE approach: relaxing the ACID properties
- **Basically Available**: The DBMS works “basically” all the time
- **Soft state**: Doesn’t have to be consistent all the time
- **Eventually consistent**: The system will eventually (i.e., finally) become consistent when the updates propagate

NoSQL DBMS categories

Four main categories:

- Key-value stores
- Column stores
- Document stores
- Graph databases

Key-value stores

- Data model: **<key, value> pairs**
- Values can contain every kind of data (no schema)
- **No queries on values (just on keys)**
- Non-primary-key-based querying:
 - Programmatically – for instance with MapReduce, a programming model for processing very large datasets

Key_1	Value_1
Key_2	Value_2
Key_3	Value_1
Key_4	Value_3
Key_5	Value_2
Key_6	Value_1
Key_7	Value_4
Key_8	Value_3

Key-Value Store

Key-value stores

- **In-memory key-value stores**
 - All the data in memory (sometimes optional durability)
 - Redis, Memcached, Oracle Coherence...
- **Persistent key-value stores**
 - Riak, BerkleyDB...

Key_1	Value_1
Key_2	Value_2
Key_3	Value_1
Key_4	Value_3
Key_5	Value_2
Key_6	Value_1
Key_7	Value_4
Key_8	Value_3

Key-Value Store

Suitable use cases

- Storing session information
 - User profiles, preferences: each user has a unique userID as well as preferences such as language, color, timezone...
- Shopping cart data
 - Shopping information stored in a custom format and associated to the userID key

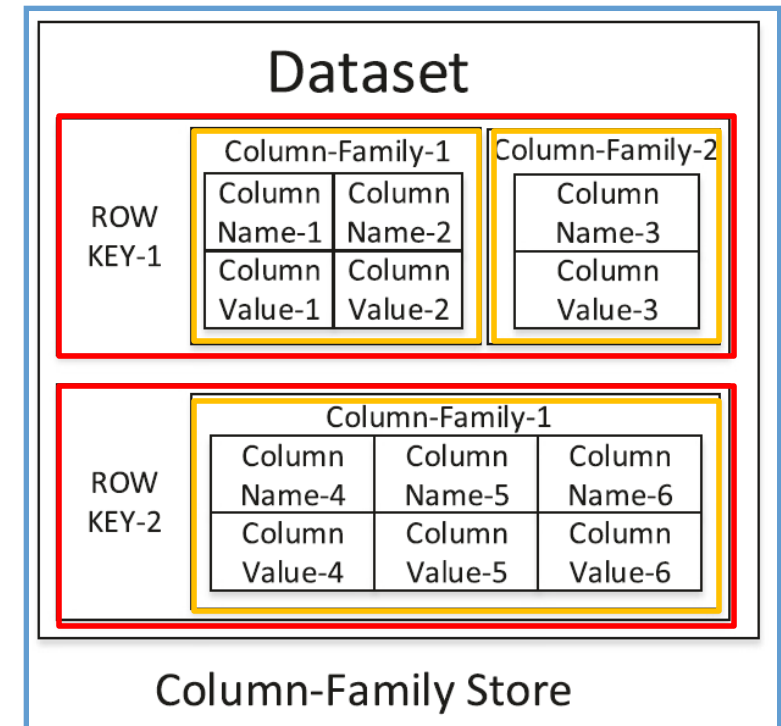


Suitable use cases

- Storing the intermediate results of computationally intensive tasks (especially in-memory stores)
 - Facebook uses Memcached for data-intensive algorithms
 - LinkedIn uses Voldemort on top of its storage system

Column stores

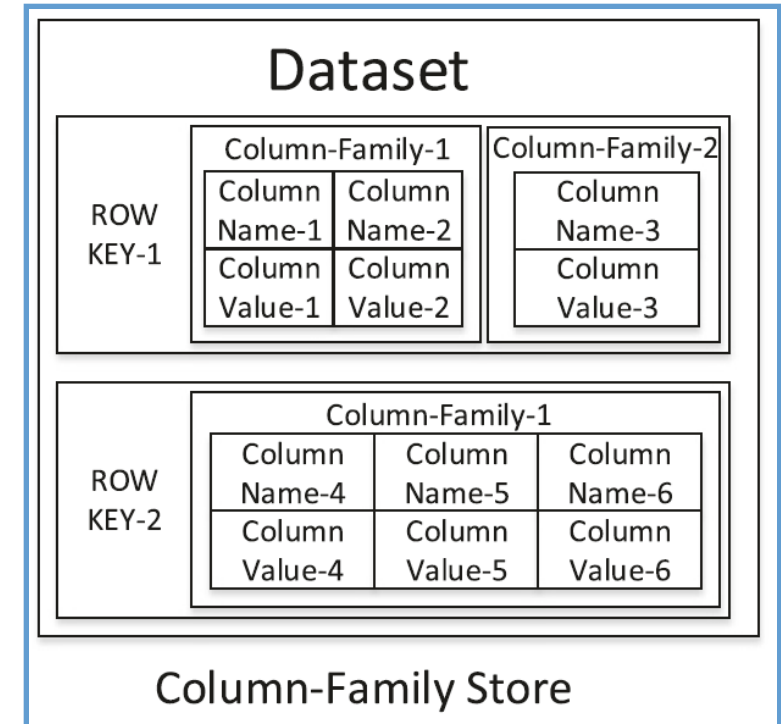
- Most of them derive from **Google BigTable**:
 - Each row has several **column families**
 - Column families are **sets of time-stamped key-value pairs**



- Different rows may have different column families

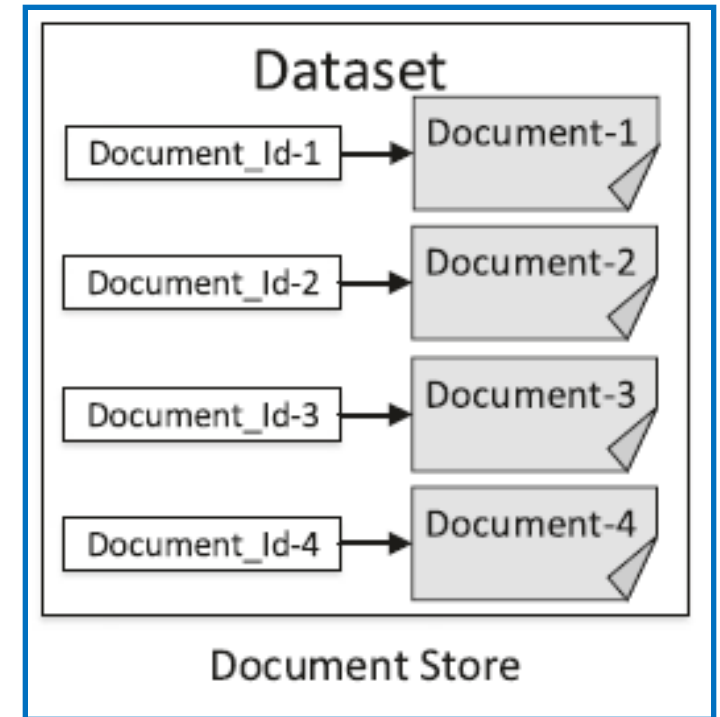
Suitable use cases

- Time-series data
- Sensor data
- Social media analytics
- But if you need to aggregate data (such as SUM or AVG queries) you must do it programmatically



Document stores

- Data model: **<key, document> pairs**
- Document: JSON, XML, or other **semistructured** formats
- Documents don't have a fixed schema
- Basic operations: as for key-value stores, but **queries also on document contents**



Document stores

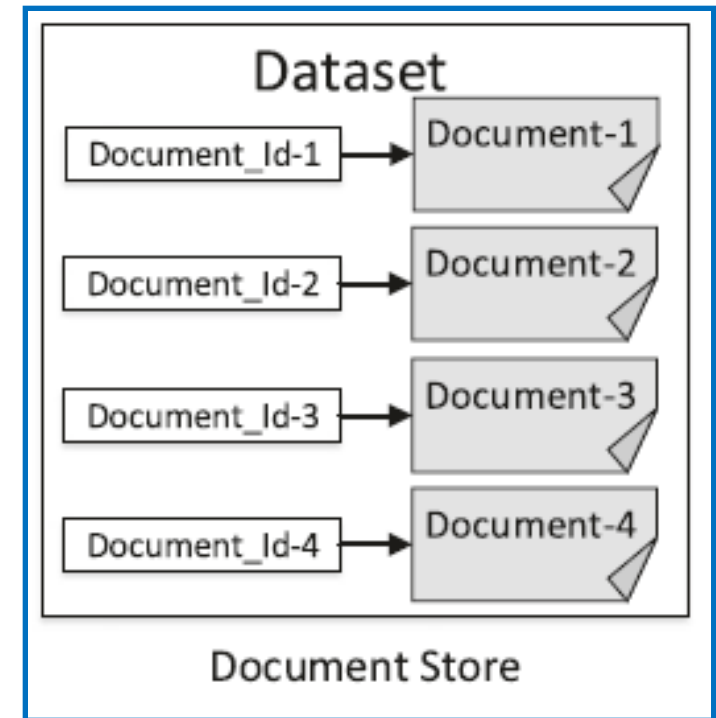
JSON document examples:

- A <name, surname> pair:

```
{"firstName":"John", "lastName":"Doe"}
```

- An array of employees:

```
{"employees":[  
  {"firstName":"John", "lastName":"Doe"},  
  {"firstName":"Anna", "lastName":"Smith"},  
  {"firstName":"Peter", "lastName":"Jones"}  
]}
```

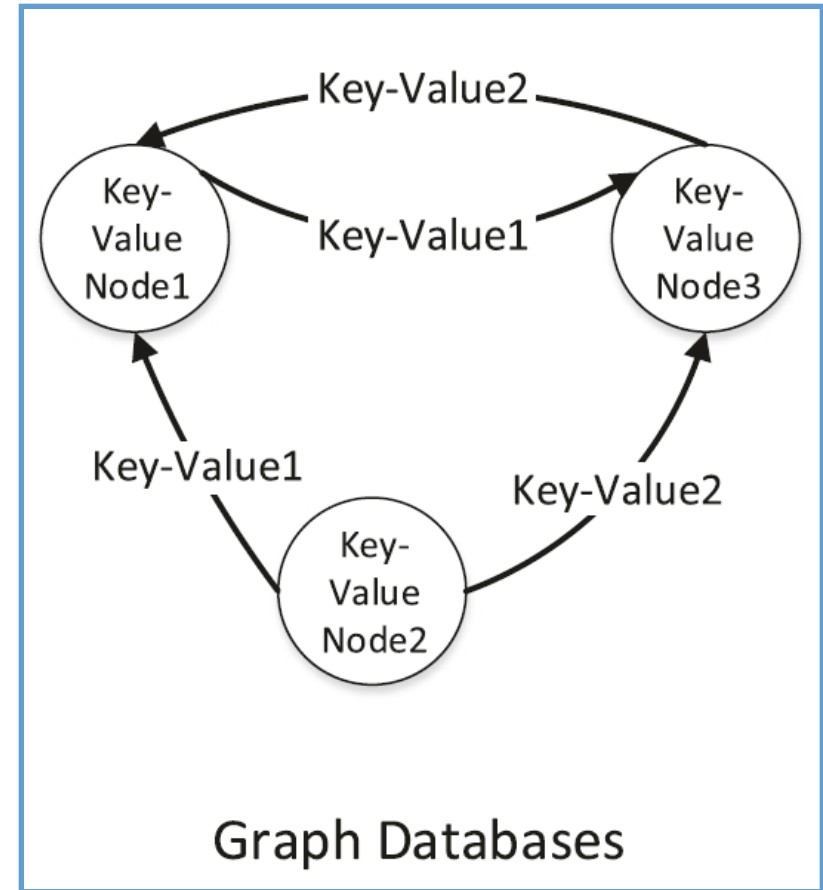


Suitable use cases

- Semi-structured data like blog posts
 - They have some fixed fields, and many variable entries
- Event logging
- Web analytics
- Real-time analysis
- E-commerce apps

Graph databases

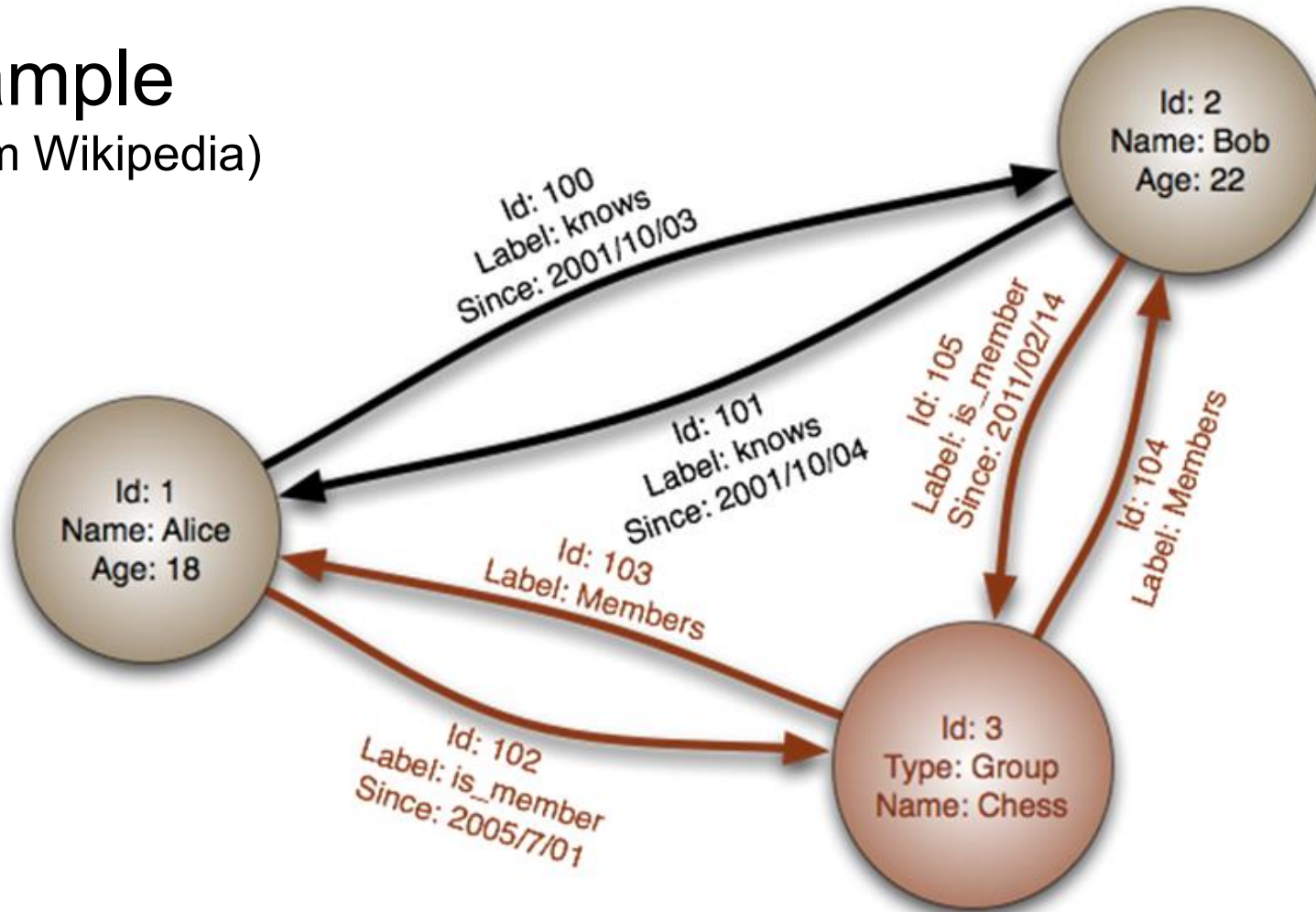
- Data model: graphs, whose **nodes and edges have properties represented as key-value pairs**
- Optimized for highly interconnected data
- Some of them (such as Neo4J) are ACID-compliant
- Other products: Titan, Graphite...



Graph databases

Example

(from Wikipedia)



Graph databases

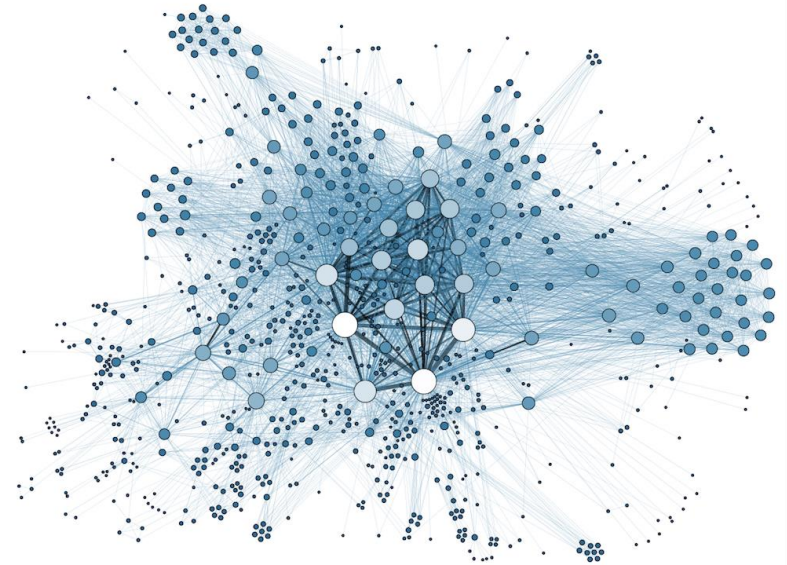
- Example **query**: "select the name of those persons who know a member of the group 'Scacchi' "

```
SELECT ?p_name  
WHERE
```

```
{  
  ?x person:Name      ?p_name;  
      person:knows    ?y.  
  ?y person:is_member ?z.  
  ?z group:Name       "Scacchi".  
}
```

Suitable use cases

- All applications relying on some kind of graphs
 - Social networks
 - Biological networks
 - Narrative networks
 - Link analysis
 - ...
- Unsuitable for other kinds of data
- Partitioning is inefficient



Querying in NoSQL

- NoSQL: Big variety of languages (no standard)
- **Efficiency**
 - Natively: simple queries on raw data
 - More complex queries: programmatically
- **Usability**
 - Many NoSQL databases support SQL-like languages

Typical NoSQL query languages

- **Key-value stores**: none (put, get, update, delete)
- **Column stores**: programmatic, or SQL-like languages (Apache Hive, Apache Pig Latin...)
- **Document stores**: proprietary, JSON-oriented
- **Graph databases**: SparQL, proprietary

Conclusions

- NoSQL: Choosing the right **tradeoff among consistency, scalability and efficiency** is not easy
 - Business **requirements may change** with time
 - If the DBMS doesn't guarantee consistency, the **application must do it**
 - **Relational DBMS can scale:**
MAIN-MEMORY DBMS store all data in RAM

Conclusions

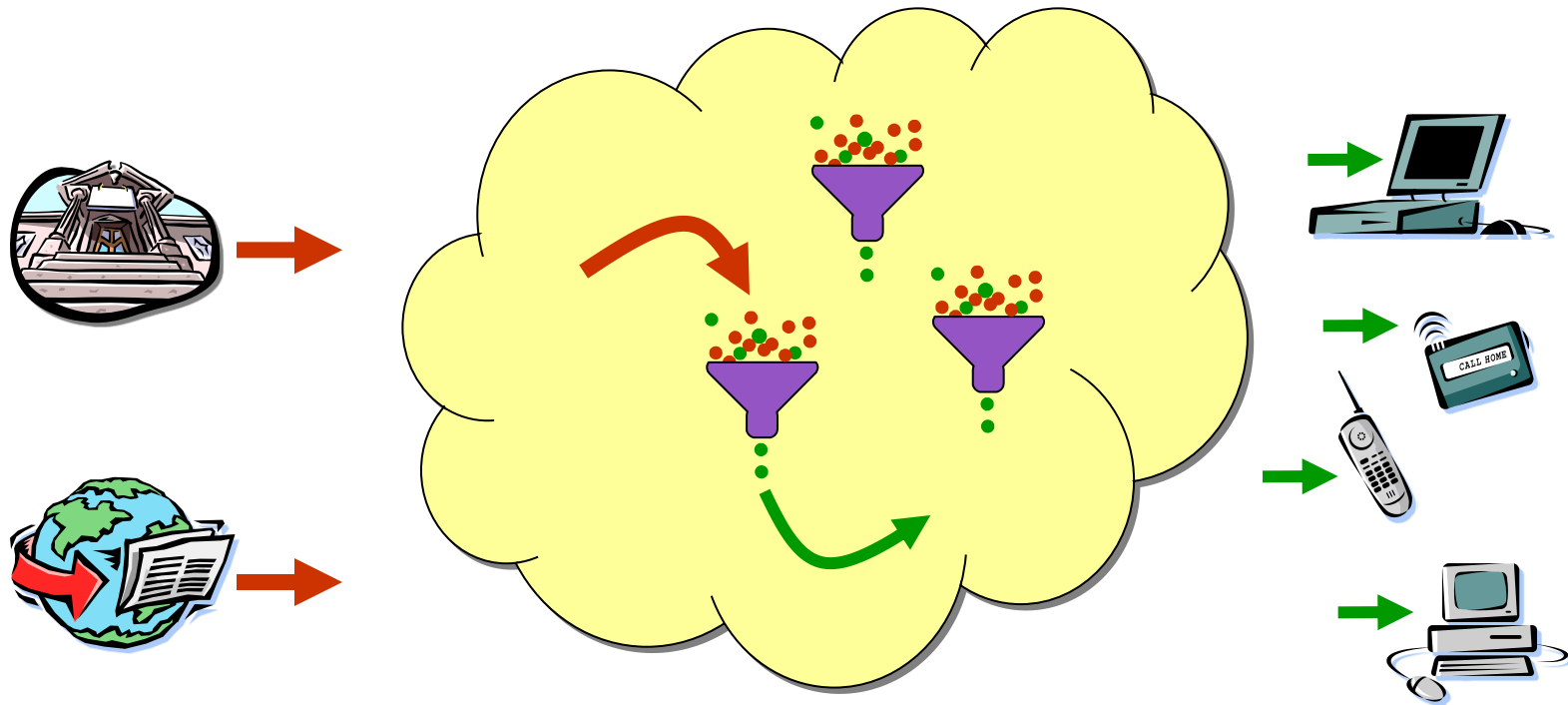
- **NoSQL**: Lack of SQL-like language makes it hard to perform queries
 - Effort is moved from “SQL” to “programming”
 - Queries not portable to other NoSQL databases

The publish/subscribe paradigm

Publishers: producers of information
(e.g., stock quote, news feeds...)

Brokers: identify events that match consumer interests

Subscribers: consumers of information

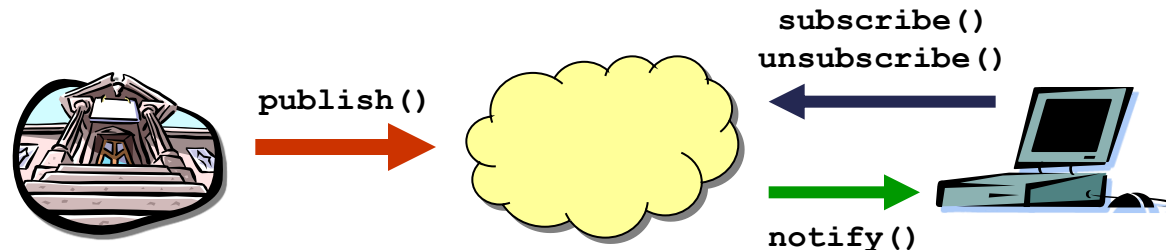


Publish/subscribe middleware

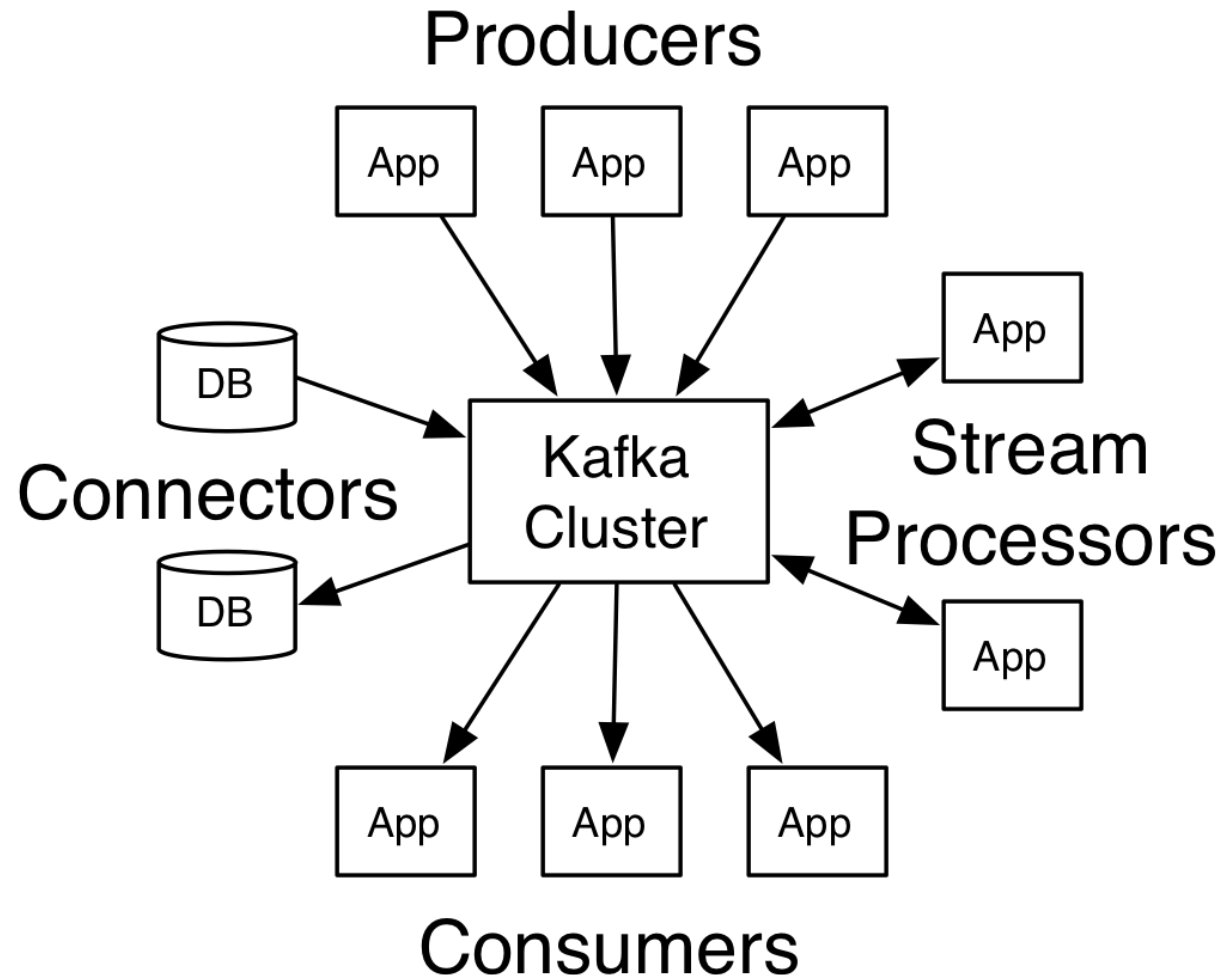
Centralized vs. distributed, persistent (DB) vs. transient,
topic- vs. content-based, reliable vs. best-effort, etc.

Publish/subscribe benefits

- **Space**: consumers and producers do not need to know each other
- **Time**: consumers and producers do not need to participate at the same time
- **Synchronization**: producers are not blocked, consumers receive messages asynchronously
- **Indirect addressing** based on data and subscriptions



Apache Kafka: publish/subscribe at scale



A data stream pipeline

